Neon Carbon — Next Steps & AI Handoff (v1)

This file captures **what works now**, **what to run**, and a **short, concrete plan for tomorrow**. It also includes an **AI handoff** section so any assistant (or future you) can instantly continue.

---

0) Current State (✅/ ✔️)

- **dex-stream-orca** — ✅builds & runs
- **normalizer-orca** — ✅builds & runs, emits normalized swap JSON
- **price-aggregator** — ✅builds & runs, emits compacted `prices.latest`
- **ui-gateway** — ✅builds & runs, WS at `ws://localhost:8081/ws/{prices,swaps,ping}`
- **ui-web** — ✅Vite dev server renders layout, mock mode works, ready to wire real WS

Topics expected:

- `orca-swaps` (raw protobuf)
- `orca-swaps.parsed` (normalized JSON)
- `prices.latest` (compacted JSON)

---

1) Runbook — Full Loop (local)

Kafka (ensure topics exist)

Create the compacted topic (once):

```
kafka-topics --bootstrap-server localhost:9092 \
  --create --if-not-exists --topic prices.latest \
  --partitions 3 --replication-factor 1 \
  --config cleanup.policy=compact
```

Start the services (separate terminals)

1) Producer

```
ENDPOINT=<geyser grpc url> \
X_TOKEN=<key> \
KAFKA_BROKER=localhost:29092 \
TOPIC=orca-swaps \
 cargo run -p dex-stream-orca
```

2) Normalizer (Orca)

```
KAFKA_BROKER=localhost:29092 \
INPUT_TOPIC=orca-swaps \
OUTPUT_TOPIC=orca-swaps.parsed \
DLQ_TOPIC=orca-swaps.dlq \
GROUP_ID=normalizer-orca-v1 \
 cargo run -p normalizer-orca
```

3) Price Aggregator

```
KAFKA_BROKER=localhost:29092 \
IN_TOPIC=orca-swaps.parsed \
OUT_TOPIC=prices.latest \
GROUP_ID=price-aggregator \
 cargo run -p price-aggregator
```

4) UI Gateway (WS bridge)

```
KAFKA_BROKER=localhost:29092 \
PRICES_TOPIC=prices.latest \
SWAPS_TOPIC=orca-swaps.parsed \
 cargo run -p ui-gateway
```

5) UI Web (Vite)

```
cd crates/ui-web
npm run dev
# open http://localhost:5173
```

Optional mock mode (for UI without backend): seed mock data via your `seedMock()` utility and/or keep the gateway offline. The UI should still render with mock frames.

---

2) Wiring UI → Live Data (today)

1. In `ui-web/src/hooks/useWS.ts`, point to gateway:
2. `ws://localhost:8081/ws/prices`
3. `ws://localhost:8081/ws/swaps`
4. Ensure `useWS()` is called inside `PricesPanel` and `SwapsPanel`.
5. Confirm WS status pill goes **green** (connected to `/ws/ping`).
6. You should see price cards populate and swaps feed scroll.

3) Frontend Tasks (tomorrow, \~2–4h)

- **Focus Drawer:** clicking a price card opens a right-side drawer with a full **Lightweight-Chart** of that pair (keep \~120 points in memory; append on update).
- **Virtualized Swaps:** use TanStack Virtual for the right column; keep the last \~1000 swaps.
- **Global Filters:** Pair selector + DEX filter in the header; persist to URL query.
- **Visual polish:** subtle glow hover, up/down coloring, tabular numerals everywhere.
- **Type safety:** factor shared `types.ts`; narrow any `any`.
- **QA:** dark tokens check — ensure `index.css` includes the Neon Carbon CSS variables and `.card` style.

---

4) Gateway/Backend Tasks (tomorrow, \~1–2h)

- **Coalesce broadcast:** batch price frames every 250ms to reduce WS chatter.
- **CORS:** allow `http://localhost:5173` (if you expose REST later).
- **Schema guards:** (optional) add `zod` in UI to validate incoming frames and drop bad ones.
- **DLQ dashboards:** consider small viewer for `orca-swaps.dlq` later.

---

5) Tooling & DX

- **ESLint/Prettier** (eslint v9 config) and `prettier-plugin-tailwindcss` installed.
- **Vite overlay checker:** `vite-plugin-checker` (ts + eslint) works; fix warnings as you go.
- **Cargo watcher** (optional): `cargo install cargo-watch` → `cargo watch -x 'run -p ui-gateway'` for hot reload.

---

6) Troubleshooting Quick Hits

- **White screen:** check `src/index.css` has the token variables & background, and Tailwind `content` globs include `src/**/*.{ts,tsx}`.
- **WS offline:** run `ui-gateway`; verify `/ws/ping` connects.
- **No prices:** confirm `prices.latest` topic exists and aggregator is running.
- **ESLint v9 errors:** use the new flat config (`eslint.config.js`) and minimal options.

---

7) AI Handoff — Context for Next Session

Project: MEVana — Solana DEX stream → normalize → price aggregation → real-time dashboard (Neon Carbon). All Rust microservices, UI in React + Tailwind.

Active services & topics:

- `dex-stream-orca` → `orca-swaps` (protobuf)
- `normalizer-orca` → `orca-swaps.parsed` (JSON)
- `price-aggregator` → `prices.latest` (JSON, compacted)
- `ui-gateway` → WS fanout of `{prices, swaps}`
- `ui-web` → reads WS, renders cards/sparklines/swaps feed

Data contracts:

- `PriceSnapshot { ts, dex, pair, price, src_sig, window_secs }`
- `NormalizedSwap { ts, slot, sig, dex, token_in?, amount_in?, token_out?, amount_out? }`

Immediate next goals:

1. Wire UI to gateway (if not already) and add the focus drawer chart.
2. Batch WS frames on gateway for smoother rendering.
3. Add filters (pair/DEX) with URL state.

Stretch: Arb radar (cross-DEX spread cards) fed from a new `arbs.found` topic.

Assumptions: local Kafka at `localhost:29092`, dev UI at `5173`, gateway at `8081`.

If stuck:

- Verify topics with `kafka-topics --describe`.
- Consume sample messages (`kafka-console-consumer`) from each topic.
- Check that `index.css` contains the Neon Carbon tokens.

End of v1 — keep this doc in `docs/` *and update the checklists as you iterate.*